claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_f1431155-4a0\agent-abd43b5796500f891.jsonl`
- SHA-256: `a523961d203fa4a68a2edd7a5767f91f5993e09118e3a1e93597c9cdf61539d3`
- Source modified: `2026-06-21T07:25:32+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_f1431155-4a0`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T07:23:57.137Z)

```
You are reviewing milestone M1 of the COMPUTE_DECOUPLED_SERVING project in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
The spec is docs/COMPUTE_DECOUPLED_SERVING/README.md (read §2 D7-D15, §4.3 "Profile selection",
and §7 the tracker row M1). M1 must be: PURE, ADDITIVE, with deployment.profile='' (the default)
== today's behaviour, ZERO core change. A *deployment profile* is opt-in and applies a coherent
preset bundle of EXISTING knobs (indexing.detector_impl/skip_ai_handoff, storage.backend) plus
the
new deployment.compute_target. Precedence: built-in defaults < profile preset < config.json <
env.
D15: auto-detect SUGGESTS, never auto-applies (cloud spend never entered silently).

Files changed (read them all, plus the existing tests/test_config.py to understand the prior
contract):
backend/config/presets.py (NEW)
backend/config/models.py (added DeploymentConfig + AppConfig.deployment field)
backend/config/loader.py (added _deep_merge, _resolve_explicit_profile, preset application)
backend/config/__init__.py (re-exports)
tests/test_config_deployment.py (NEW)

Ground EVERY claim in the actual code with a file:line citation. Do NOT speculate. If you assert
a
bug, give the exact input that triggers it and the wrong-vs-right output. Rate confidence
honestly;
prefer FEWER, HIGH-CONFIDENCE findings. A clean bill ([] findings) is a valid and good result.

LENS: spec & D15 adherence. Compare the implementation to README §4.3 and §7 M1. Verify:
(a) auto-detect SUGGESTS but the loader NEVER auto-applies a profile from the environment (a
bare
K_SERVICE / CI / cuda box must NOT silently select a profile); (b) the presets only touch knobs
that
EXIST today and do NOT pre-empt gated milestones ? specifically they must NOT set input_backend
(M2) or
flip the output/workspace base (storage.single_folder_per_video / output base ? M3, Launch-
Report-gated);
(c) the preset values are coherent with the decisions (cloud-serving keeps Gemini ON per D11;
truly-local
is zero-cloud; cpu-mock is stub+no-AI); (d) compute_target is inert/recorded-only in M1 (nothing
in the
core branches on it). Flag any over-reach beyond M1 scope or any decision contradiction.
```

2. assistant (2026-06-21T07:23:59.738Z)

```
I'll review M1 of the COMPUTE_DECOUPLED_SERVING project. Let me start by reading the spec and
```

the changed files.

3. user (2026-06-21T07:24:00.955Z)

Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage

> **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** · **Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory `current-state` + `docs/NEXT_STEPS.md` once work starts.
> **Relation to existing docs:** the productionization successor to the archived `DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from [PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
> **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed **compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an afterthought.
> **Honesty note:** claims tagged `[verified]` (checked against code by the design workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.

> **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13, a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user *confirms* (D15).

0. TL;DR
The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB / mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local** (local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive / bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus **cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the **parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.

1. Problem & goal
- **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4) and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero + cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local mode, with the **core (detect+stitch) unaffected** by which one runs.
- **The two user-facing outcomes** (owner, 2026-06-21):
  - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from **gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching** (stitching is compute-intensive ? run it where its cost is accounted for).
  - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local GPU**, run inference **and** stitching **completely locally**.
- **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows + stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup choice**, never a code branch in the core.

2. Decisions locked

#	Decision	Source	Implication
D1	The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**.	design `[verified]`	All profile difference lives in config + the two registries.

D2	**Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`.	ADR-014	Realizes PLATFORM_ARCHITECTURE's `local|hosted|byo_worker`.
D3	**Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred.	ADR-014	Enum/seam reserved so BYO isn't precluded.
D4	In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger.	owner (a)	Stitch-on-laptop would hide real cost.
D5	**Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU.	deploy report 2026-06-20	Cross-GPU asserted at the rally/window level, never CSV bytes.
D6	**Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report.	[[launch-report-convention]]	Zero-regression discipline.
D7	**Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing).	owner 2026-06-21	M6/M7. Revisit a warm lane only on a UX complaint.
D8	**Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change.	owner 2026-06-21	M8.
D9	**Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header).	owner 2026-06-21	M9. One identity system.
D10	**Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA.	owner 2026-06-21	M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel.
D11	**Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.)	owner 2026-06-21	M7 + D12.
D12	**Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off).	owner 2026-06-21	new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness.
D13	**Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime").	owner 2026-06-21	Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs.
D14	**NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**.	owner 2026-06-21	Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it).
D15	**Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced.	owner 2026-06-21	Amends §4.3 (auto-detect = a proposed default, not an auto-decision).

3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*

This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:

ADR / doc	Contributed	Subtlety carried into this project
ADR-005	Permissive licensing (MIT/Apache-2.0/BSD only)	The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean.
ADR-008	Owned-model topology; **train==serve featurizer single-sourcing**	The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**.
ADR-009	KhelSutra Guru; **local-first delivery**	The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class.
ADR-010	DECOUPLED: DO?GCP pivot	The cloud-compute origin (GPU + co-located bucket).
ADR-011	DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture	**This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**.
ADR-012	GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever	The serving substrate + the 7-step seed scope this expands.
ADR-013	Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only	The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target.
? ADR-014 (new)	**Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered	This doc. **Refines** ADR-011/012, does **not** supersede them.
PLATFORM_ARCHITECTURE.md	The `ComputeBackend`/`StorageBackend` registries	The abstraction this realizes in code.
VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md	L0 fully-local tier / `LocalDesktop` backend	The truly-local profile = these, productionized.
07-COMPUTE-ABSTRACTION (archived)	`DeviceContext`/`DeviceProfile` (designed)	WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes.
DEPLOY_REPORT_GCP_2026-06-20 (archived)	First real L4 run; **cross-GPU sub-pixel parity finding**	Why D5 (parity at window level, not CSV bytes).

4. Design / architecture
See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**

4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.

Non-goal (the trap to avoid): *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).

4.2 The deployment profiles
Profile	Compute	Storage	Orchestration	When
truly-local	`compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process**	`local` (or `gdrive` = mounted Drive)	`python -m backend.main` or `worker infer` CLI; existing async job worker	Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b)
cloud-serving	`compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job**	`gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL	Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero	Hosted SaaS ? owner (a)

| **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
| *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |

4.3 Profile selection (the "startup/setup config") `[design]`
ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
- `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
- Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
- **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
- **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
- The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.

4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)

4.5 Reuse map ? most of the seams already exist `[verified]`
Exists: the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.

4.6 The data-flywheel harness (D12 / M11) `[design]`
Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:
1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.

6. Honest limits ? what this does NOT do / prove
- A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
- **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
- The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
- This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.

7. Deliverables & progress tracker ? **source of truth**
Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.

ID	Deliverable	Depends	Gated?	Status	PR
M0	This design doc + project dir + `runlogs/` + ADR-014	?	owner sign-off	?	**? APPROVED 2026-06-21**
M1	`deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default)	M0	safe	?	?
M2	`input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests	M1	safe	?	?
M3	Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical	M1	? default-OFF + Launch Report on flip	?	?
M4	`DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring	M1	safe (cuda default unchanged)	?	?
M5	**truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run)	M2,M3,M4	owner real-GPU	?	?
M6	Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests	M5	? owner (GCP spend)	?	?
M7	**cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet)	M6	? owner (spend)	?	?
M8	Per-job **cost ledger** (v_ migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost`	M7	? owner (billing)	?	?
M9	**Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring	M7	? owner (privacy/counsel)	?	?
M10	`byo_worker` / zero-infra-BYO ? **design-only, DEFERRED**	M8,M9	? explicit owner approval	?	?
M11	**Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9).	M7,M9	? owner (consent/privacy)	?	?
M12	**`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC.	M7,M9	? owner (OAuth/consent)	?	?

Testing strategy is a first-class deliverable ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).

8. Open questions ? owner *(blocking-gates vs nice-to-knows)*

```
130	**? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
131	1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
132	3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
133	4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
134	5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
135	6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).
136
137	**Still open (non-gating, smaller):**
138	2. **Stitch placement:** always co-located in the detect job (simplest, one metered unit) vs split to a cheap CPU Cloud Run service when bursty? *Default co-located unless metrics say otherwise.*
139	7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
140	8. **NVDEC/NVENC** in the Cloud Run image (cut stitch wall-time/cost) vs libx264 (determinism/parity)?
141	9. **Input cap:** is `<2GB` a hard client reject, or accept to the 4GB upload cap and route larger files elsewhere?
142	10. **Directory name:** `COMPUTE_DECOUPLED_SERVING/` (chosen) vs keep `CLOUD_RUN_GPU_SERVING/` ? confirm.
143
144	## 9. Definition of done
145	- [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
146	- [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
147	- [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
148	- [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth (M9) gates multi-tenant.
149	- [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented adult jobs; the owned model is on a measured path to the D11 floor.
150	- [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
151
152	## 10. References
153	**ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014** (`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`, `HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`, `LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:** `docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`, `stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`, `backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`, `backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-mappers ? synthesis; 58 findings).
154
155	---
156	### Changelog
157	- 2026-06-21 ? created from the design workflow output; M0 in progress.
158
```

4. assistant (2026-06-21T07:24:03.085Z)

Now let me read the changed files.

5. user (2026-06-21T07:24:04.045Z)

```
1	"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).
2
3	A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
```

```
config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

Precedence (lowest ? highest), realised by the loader::

    built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set

This module is **pure data + pure functions** ? no IO, and (critically) **no torch import at
module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
(``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15: cloud
spend is never entered silently).
"""

from __future__ import annotations

import copy
import os
from typing import Any, Mapping, Optional

# Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` mirrors
# these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")

# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
# when config.json explicitly overrides compute_target into a state inconsistent with the profile.
COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
    "truly-local": "local_gpu",
    "cloud-serving": "cloud_run",
    "cpu-mock": "cpu_mock",
}

# A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs
# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}`` plus
# ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones extend
# these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
# Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
PROFILE_PRESETS: dict[str, dict[str, Any]] = {
    "truly-local": {
        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
        "indexing": {"skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
    "cloud-serving": {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
        "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
        "storage": {"backend": "gcs"},
    },
    "cpu-mock": {
        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
```

```
        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
        "storage": {"backend": "local"},
    },
}


def is_known_profile(profile: str) -> bool:
    """True for a non-empty, recognised profile name."""
    return bool(profile) and profile in PROFILE_PRESETS


def preset_for(profile: str) -> dict[str, Any]:
    """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
    module-level table), or ``{}`` for ``""`` / an unknown profile."""
    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


def _truthy(val: Optional[str]) -> bool:
    """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
    if val is None:
        return False
    return val.strip().lower() in {"1", "true", "yes", "on"}


def _cuda_available() -> bool:
    """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
    ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
    try:
        # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
        # and optional). Only reached when a caller opts into probe_cuda.
        import torch

        return bool(torch.cuda.is_available())
    except Exception:
        return False


def suggest_profile(
    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
) -> str:
    """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.

    Signals, in priority order:

      - ``K_SERVICE`` set (Cloud Run)         ? ``"cloud-serving"``
      - ``CI`` truthy                         ? ``"cpu-mock"``
      - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
      - otherwise                             ? ``""`` (no strong suggestion)

    ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and never
    imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe passes
    ``probe_cuda=True``. The result is advisory only: the caller decides whether to act on it.
    """
    e = os.environ if env is None else env
    if e.get("K_SERVICE"):
        return "cloud-serving"
```

```
    if _truthy(e.get("CI")):
        return "cpu-mock"
    if probe_cuda and _cuda_available():
        return "truly-local"
    return ""

```

6. user (2026-06-21T07:24:04.523Z)

```
"""Pydantic models for the application configuration.

This module defines the canonical, validated, typed configuration for the
Badminton Highlight Indexer. Built-in defaults live here (single source of truth).

The models support graceful migration from old config.json files via extra="allow".
"""

from __future__ import annotations

from typing import Any, List, Literal, Optional

from pydantic import BaseModel, Field, field_validator, model_validator

Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    relevance: ValidationRelevanceConfig = Field(default_factory=ValidationRelevanceConfig)


class TrackNetConfig(BaseModel):
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""
    queue_length: int = 5
    stage_in_wsl: bool = True
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
    # decoded frame cache) are deleted automatically ? they are pure intermediates,
    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
    keep_frames: bool = False
    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
    # 0 = disabled.
    min_free_gb: float = 0.0


class WasbIndexConfig(BaseModel):
    """Typed config for the live WASB shuttle substrate (indexing.wasb).
```

```

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
        # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
        device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
        python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
        # from the source. Set true only for debugging. On FAILURE they are kept so a re-run resumes.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 = disabled.
        min_free_gb: float = 0.0
        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs for a
        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical to the
        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream == WSL disk,
        # 1194/1195 frames byte-identical; native run-to-run byte-identical / deterministic).
        # Tri-state:
        #   None       = per-runner default ? the WSL runner keeps DISK extraction (unchanged Windows
        #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING (the perf win).
        #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2 can't seek).
        stream_video: Optional[bool] = None


    class FusionConfig(BaseModel):
        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

        Every default reproduces control behaviour: the fusion segmenter persists the
        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
        true ? the person-cue features, but it does NOT gate the served handoff unless a
        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
        supplied ? off-court persons (spectators / adjacent courts) are excluded.
        """
        # Which trajectory substrate seeds the windows (shuttle stays primary).
        substrate: Literal["wasb", "tracknet"] = "wasb"
```

```
    # Windowing velocity threshold for the fusion family (the engine reads
    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
    velocity_threshold: float = 0.02
    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
    # enabled ? so the default path never imports torch / touches the GPU.
    cue_flags: dict[str, bool] = Field(default_factory=dict)
    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-scoring).
    min_crossings: int = Field(default=0, ge=0)
    # Served Gate B: when the person cue is on, drop windows with median in-court players
    # < this. 0 = OFF.
    min_players: int = Field(default=0, ge=0)
    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
    court_polygon: Optional[list[list[float]]] = None
    # Net line for the person two-sided-motion split (person features only ? the shuttle
    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
    net_axis: Literal["x", "y"] = "y"
    net_position: float = Field(default=0.5, ge=0.0, le=1.0)


class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
    # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
    # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
    # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
    # RALLY_DETECTOR_IMPL env var overrides this. See pipeline/detectors/{stub,native_wasb}_runner.py.
    detector_impl: str = "auto"
    motion_threshold: float = 0.08
    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0
    chunk_duration: float = 90.0
    chunk_overlap: float = 10.0
    detector_model: str = "fasterrcnn_resnet50_fpn"
    detector_score_threshold: float = 0.5
    yolo_velocity_threshold: float = 0.15
    yolo_frame_skip: int = 3
    yolo_tracker: str = "bytetrack.yaml"
    yolo_prefetch_workers: int = 2
    min_action_window_duration: float = 2.0
    # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0 = a window
    # longer than this many seconds is recursively split at its largest internal inactivity gap (?
    # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't swallow
    # several rallies + the dead time between them. Never merges, only cuts (recall can't drop).
    # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip. Measured n=13
    # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0, p=0.0005; cap=6
    # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate 0.651?0.161). ***
    max_window_duration: float = 6.0
    window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split point
    # HARD-CAP fallback for W1: when True, an over-long window with NO internal inactivity gap
```

```
        # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob) is
        # chopped at its midpoint until ? max_window_duration, making the cap a true upper bound.
        # Only cuts (recall can't drop). OFF by default until measured/promoted.
        window_hard_cap: bool = False
        # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving slowly
        # (picked up / walked) above velocity_thresh, so the window over-extends its END (the measured
        # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose trailing
        # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only cuts.
        # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but significant
        # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13). The W1 cap
        # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md "R1". ***
        window_inactivity_end: float = 1.5
        inactivity_rally_thresh: float = 0.20
        inactivity_min_density: float = 0.30
        # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active blob: after
        # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group STILL longer
        # than window_blob_factor × max_window_duration (no internal gap, no rally-end signal ? e.g.
        # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as a FORCED cut
        # (logged + flagged on the window) so the degenerate single-window outcome is avoided and the
        # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from window_hard_cap
        # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER R4 and only
        # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
        window_blob_fallback: bool = False
        # Magnitude gate that keeps R5 surgical: only a group longer than this × max_window_duration is a
        # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual is ~11×).
        # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-active
        # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md "R5".
        window_blob_factor: float = 4.0
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff entirely and
        # emit their local candidate windows AS the rallies ? no provider, no API key, nothing
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
```

```
    # path, or to operate with zero cloud dependency). The pure `gemini` segmenter ignores this.
    skip_ai_handoff: bool = False
    # Optional debug knob: trim every video to the first N seconds before processing.
    debug_duration: Optional[float] = None

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
    fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod
    def segmenter_must_be_registered(cls, v: str) -> str:
        # Registry check happens at runtime in main/segmenter selection.
        # Here we just allow any string for forward compatibility.
        return v

    @model_validator(mode="after")
    def _chunk_step_must_be_positive(self) -> "IndexingConfig":
        # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-positive
        # step makes `while t < duration: t += step` loop forever, growing memory before any
        # GPU work. Reject the bad config at load time instead.
        if self.chunk_overlap >= self.chunk_duration:
            raise ValueError(
                f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration "
                f"({self.chunk_duration}); otherwise chunking never advances."
            )
        return self


class OutputConfig(BaseModel):
    default_highlights_name: str = "highlights.mp4"
    db_name: str = "sports_indexer.db"
    # Directory where the DB, highlight reels, and processed clips are written.
    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
    output_dir: str = "output"


class WatermarkConfig(BaseModel):
    """Branding overlay burned into each highlight clip during the per-clip re-encode
    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
    (under `stitching.watermark` in config.json) to turn it off. The text is fully
    configurable. The concat stage stays stream-copy (-c copy)."""
    enabled: bool = True
    text: str = "Generated by Khelsutra.guru"
    logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
    font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family below
    font: str = "Sans"         # fontconfig family used when font_path is empty
    font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a fraction of frame height
    opacity: float = Field(default=0.85, ge=0.0, le=1.0)
    box: bool = True           # faint backing box behind the text for legibility
    margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
    position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-right"


class StitchingConfig(BaseModel):
    begin_padding: float = 0.5
    end_padding: float = 0.5
```

```
    use_cache: bool = False
    min_shot_count: int = 1
    # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose duration
    # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
    # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
    # fully-local no-AI path, whereas duration is derived from the rally's own start/end. The
    # local detector over-fires and its false positives are mostly SHORT (motion blips,
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
    watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


class LoggingConfig(BaseModel):
    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
    # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
    # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
    # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
    # them to WARNING; set true to restore full chatter for request-flow debugging.
    verbose_http: bool = False


class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]


class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO Spaces
    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a locally MOUNTED
    Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""
    # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
    # One folder per video keyed by the MD5 video_id: <root>/[<workspace_prefix>/]<video_id>/...
    # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the root
    # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
```

```
When OFF
    # the legacy flat layout is used unchanged.
    single_folder_per_video: bool = False
    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
    workspace_root: str = ""
    # Cloud namespace prefix under the root so per-video folders sit tidily beside
    # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root directly.
    workspace_prefix: str = "workspaces"
    local: dict[str, Any] = Field(default_factory=dict)
    s3: dict[str, Any] = Field(default_factory=dict)
    gcs: dict[str, Any] = Field(default_factory=dict)
    gdrive: dict[str, Any] = Field(default_factory=dict)


class DeploymentConfig(BaseModel):
    """Deployment profile ? one switch that selects a coherent bundle of compute + storage
    knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).

    **Pure / additive:** the empty default profile (``""``) applies NO preset, so the pipeline
    behaves exactly as before this section existed. A profile is opt-in (``config.json``
    ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
    *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core reads
    these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING knobs
    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.

    Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""

    # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
    profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
    # Where the core runs. Set by the profile preset; overridable in config.json. Inert in M1
    # (recorded for observability; consumed by the ComputeBackend seam from M2+).
    compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""


class AppConfig(BaseModel):
    """Root configuration object.

    All runtime configuration should be accessed via an instance of this class
    (or the loader) rather than raw dict lookups.
    """

    sport: Sport = "badminton"
    ai_provider: str = "gemini"
    # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable flagship;
    # the alias never deprecates underneath us. Eval reports record the run date alongside.
    ai_model: str = "gemini-flash-latest"

    validation: ValidationConfig = Field(default_factory=ValidationConfig)
    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
    output: OutputConfig = Field(default_factory=OutputConfig)
    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
    security: SecurityConfig = Field(default_factory=SecurityConfig)
    logging: LoggingConfig = Field(default_factory=LoggingConfig)
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
```

```

    # Allow extra keys during transition so existing config.json files do not break.
    model_config = {
        "extra": "allow",
        "validate_assignment": True,
    }

    # Temporary migration shim for legacy dict-style access (config.get("section", {}).get(...)).
    # Will be removed once all call sites (validators, segmenters, etc.) move to typed
    # attribute access / the typed config. A nested *section* is returned as a plain dict
    # via model_dump, so legacy chained access keeps working ? returning the Pydantic
    # section model instead breaks it (a model has no .get), which bit every validator on
    # real runs. Scalar leaves are returned as-is.
    def get(self, key: str, default: Any = None) -> Any:
        _MISSING = object()
        val: Any = _MISSING
        if key in type(self).model_fields or hasattr(self, key):
            val = getattr(self, key, _MISSING)
        if val is _MISSING:
            # Legacy convenience: find a leaf key inside any nested section. Sections are
            # discovered dynamically from the model fields, so a future top-level section
            # (e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
            for fname in type(self).model_fields:
                section = getattr(self, fname, None)
                if isinstance(section, BaseModel) and hasattr(section, key):
                    val = getattr(section, key)
                    break
        if val is _MISSING:
            return default
        if isinstance(val, BaseModel):
            return val.model_dump(mode="json")
        return val

    def __getitem__(self, key: str) -> Any:
        val = self.get(key)
        if val is None and key not in type(self).model_fields:
            # Re-raise like a real dict for unknown top-level
            raise KeyError(key)
        return val


# Convenience type alias for callers that want to annotate
Config = AppConfig

```

7. user (2026-06-21T07:24:05.100Z)

```
"""Configuration loader and saver.

Provides a single place to load validated AppConfig with built-in defaults
merged with optional file overrides (config.json).

This replaces scattered json.load + .get() patterns throughout the codebase.
"""

from __future__ import annotations

import json
import logging
```

```python
import os
from pathlib import Path
from typing import Any

from pydantic import BaseModel

from .models import AppConfig
from .presets import (
    COMPUTE_TARGET_FOR_PROFILE,
    PROFILE_PRESETS,
    is_known_profile,
    preset_for,
    suggest_profile,
)

logger = logging.getLogger(__name__)

DEFAULT_CONFIG_PATH = Path("config.json")


def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
    """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge, scalars
    replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
    overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-profile
    path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
    out = dict(base)
    for key, val in override.items():
        if isinstance(val, dict) and isinstance(out.get(key), dict):
            out[key] = _deep_merge(out[key], val)
        else:
            out[key] = val
    return out


def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
    """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if none
    is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so a bare
    environment never silently selects a profile (and never silently spends on cloud)."""
    env_profile = os.environ.get("DEPLOYMENT_PROFILE")
    if env_profile and env_profile.strip():
        return env_profile.strip()
    dep = file_data.get("deployment")
    if isinstance(dep, dict):
        prof = dep.get("profile")
        if isinstance(prof, str) and prof.strip():
            return prof.strip()
    return ""


def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str = "") -> list[str]:
    """Dotted paths in `data` that the typed config will not honor.

    Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
    kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
    typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
    user's intent is lost ? collect every such key so load_config can warn.
    """
    unknown: list[str] = []
    fields = model_cls.model_fields
```

```
    for key, val in data.items():
        if key not in fields:
            unknown.append(prefix + key)
            continue
        ann = fields[key].annotation
        if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann, BaseModel):
            unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
    return unknown


def _get_builtin_defaults() -> dict[str, Any]:
    """Return a plain dict of the current built-in defaults.

    The AppConfig model is the single source of truth for defaults.
    """
    return AppConfig().model_dump(mode="json")


def load_config(path: str | Path | None = None) -> AppConfig:
    """Load configuration with the following precedence:

    1. Built-in defaults defined in the Pydantic models.
    2. Values from the JSON file (if present and readable).
    3. (Future) environment / CLI overrides.

    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
        try:
            with open(cfg_path, "r", encoding="utf-8") as f:
                file_data = json.load(f)
        except Exception as exc:
            # Non-fatal: continue with defaults + whatever we could parse.
            # In production we might log here.
            logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

    # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`, `null`) would
    # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and any
    # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn, per the
    # loader's "bad input is non-fatal" contract.
    if not isinstance(file_data, dict):
        logger.warning("%s is not a JSON object (got %s); ignoring it and using defaults.",
                       cfg_path, type(file_data).__name__)
        file_data = {}

    if file_data:
        unknown = _unknown_key_paths(file_data, AppConfig)
        if unknown:
            logger.warning(
                "[CONFIG WARNING] %s contains keys the typed config does not "
                "recognize (typo'd or removed knobs ? top-level extras are kept "
                "but unread; unknown section keys are silently dropped): %s",
                cfg_path, ", ".join(sorted(unknown)),
            )

```

```
    # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied at
    # consumption sites downstream). A *deployment profile* is opt-in ? it is applied ONLY when
    # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile). With no
    # profile the merge is byte-identical to the pre-M1 loader, so the default path is unchanged.
    defaults = _get_builtin_defaults()
    profile = _resolve_explicit_profile(file_data)

    if profile and not is_known_profile(profile):
        # An unrecognised EXPLICIT profile is loud, not silent (the loader's "dropped config"
        # ethos): warn and apply no preset. A bad value coming from config.json additionally
        # hits the typed Literal at model_validate below (a hard, clear error); a bad value
        # coming only from the env override is ignored here with this warning.
        logger.warning(
            "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no preset.",
            profile, ", ".join(PROFILE_PRESETS),
        )
        profile = ""

    if profile:
        merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
        # Record the profile actually applied ? the env may have chosen it over config.json,
        # so re-stamp it onto the merged deployment section to keep the record honest.
        merged.setdefault("deployment", {})
        if isinstance(merged["deployment"], dict):
            merged["deployment"]["profile"] = profile
            ct = merged["deployment"].get("compute_target")
            canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
            if ct and canonical and ct != canonical:
                logger.warning(
                    "[CONFIG] deployment profile %r normally uses compute_target=%r but "
                    "config.json overrides it to %r ? honoring the explicit override.",
                    profile, canonical, ct,
                )
            logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
                        profile, merged["deployment"].get("compute_target"))
    else:
        # No profile ? the original shallow merge (Pydantic re-defaults nested sections).
        merged = {**defaults, **file_data}
        suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only (D15)
        if suggestion:
            logger.debug(
                "[CONFIG] no deployment.profile set (manual knobs). Environment suggests %r ? "
                "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect never "
                "auto-applies ? D15).", suggestion,
            )

    return AppConfig.model_validate(merged)


def save_default_config(path: str | Path | None = None) -> Path:
    """Write a fresh default config.json.
```

```

    Used by scripts/doctor.py diagnostics and the --setup CLI path.
    Overwrites if the file exists.
    """
    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
    cfg = AppConfig()
    cfg_path.parent.mkdir(parents=True, exist_ok=True)
    cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
    return cfg_path


# Backwards-compatible alias used by some older call sites during transition.
def get_default_config() -> dict[str, Any]:
    """Return defaults as a plain dict (for transitional use only)."""
    return _get_builtin_defaults()

```

8. user (2026-06-21T07:24:05.509Z)

```
"""Configuration package.

Public API:
    from backend.config import load_config, AppConfig, Config

This package is the canonical place for all configuration access.
"""

from .loader import load_config, save_default_config
from .models import (  # re-export common ones
    AppConfig,
    Config,
    DeploymentConfig,
    IndexingConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)
from .presets import PROFILE_PRESETS, suggest_profile

__all__ = [
    "load_config",
    "save_default_config",
    "AppConfig",
    "Config",
    "DeploymentConfig",
    "IndexingConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
]

```

9. user (2026-06-21T07:24:05.943Z)

```
"""Tests for the deployment-profile config (COMPUTE_DECOUPLED_SERVING M1).

What this pins:
- The default profile ("") applies NO preset ? the loaded config is byte-identical to the
  pre-M1 default, so the core is unaffected (the project's "default-OFF, zero core change").
- Each profile applies its coherent preset bundle (compute_target + the existing
  detector_impl / skip_ai_handoff / storage.backend knobs).
```

```
- Precedence: built-in defaults < profile preset < config.json (a file sub-key overrides a
  preset sub-key while sibling preset/default sub-keys survive ? i.e. real deep-merge layering).
- D15: auto-detect SUGGESTS but never auto-applies (a bare cloud-like env does NOT silently
  select a profile / spend on cloud).
- The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
"""

import json
import logging
import os

import pytest
from pydantic import ValidationError

from backend.config import AppConfig, DeploymentConfig, load_config, save_default_config
from backend.config.presets import (
    COMPUTE_TARGET_FOR_PROFILE,
    COMPUTE_TARGETS,
    PROFILE_PRESETS,
    PROFILES,
    is_known_profile,
    preset_for,
    suggest_profile,
)

NONEXISTENT = "/nonexistent/path/12345/deployment_config.json"


def _write(tmp_path, data) -> str:
    p = tmp_path / "cfg.json"
    p.write_text(json.dumps(data))
    return str(p)


# ---------------------------------------------------------------------------------
# Defaults / additivity ? the default path must be unchanged.
# ---------------------------------------------------------------------------------
def test_deployment_defaults_are_empty_and_inert():
    cfg = load_config(NONEXISTENT)
    assert isinstance(cfg.deployment, DeploymentConfig)
    assert cfg.deployment.profile == ""
    assert cfg.deployment.compute_target == ""


def test_default_path_applies_no_preset(monkeypatch):
    """With no profile selected, the existing knobs keep their built-in defaults (no preset
    mutates detector_impl / skip_ai_handoff / storage.backend)."""
    monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
    cfg = load_config(NONEXISTENT)
    assert cfg.indexing.detector_impl == "auto"
    assert cfg.indexing.skip_ai_handoff is False
    assert cfg.storage.backend == "local"


def test_default_config_dump_only_adds_deployment_section():
    """Adding the section must not perturb any other default ? the dump equals a fresh
    AppConfig()'s dump, and the new section is present with empty values."""
    dumped = load_config(NONEXISTENT).model_dump(mode="json")
    assert dumped == AppConfig().model_dump(mode="json")
    assert dumped["deployment"] == {"profile": "", "compute_target": ""}

```

```python

# ---------------------------------------------------------------------------------
# Parity: the model Literal must match the presets table (no drift).
# ---------------------------------------------------------------------------------
def test_profile_literal_matches_presets():
    import typing

    literal_profiles = set(typing.get_args(DeploymentConfig.model_fields["profile"].annotation))
    assert literal_profiles == {"", *PROFILES}
    assert set(PROFILE_PRESETS) == set(PROFILES)
    assert set(COMPUTE_TARGET_FOR_PROFILE) == set(PROFILES)


def test_compute_target_literal_matches_presets():
    import typing

    literal_targets = set(typing.get_args(DeploymentConfig.model_fields["compute_target"].annotation))
    assert literal_targets == {"", *COMPUTE_TARGETS}
    assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)


def test_each_preset_only_touches_existing_knobs():
    """A preset must only set knobs that exist on the typed models ? otherwise it would be a
    silent no-op. Validates the merged result against AppConfig for every profile."""
    for profile in PROFILES:
        # A preset's value set must validate cleanly when merged onto defaults.
        merged = {**AppConfig().model_dump(mode="json")}
        for section, kv in preset_for(profile).items():
            merged.setdefault(section, {})
            if isinstance(merged[section], dict):
                merged[section].update(kv)
        AppConfig.model_validate(merged)  # raises if a preset key/value is invalid


# ---------------------------------------------------------------------------------
# Per-profile preset application.
# ---------------------------------------------------------------------------------
def test_cpu_mock_profile(tmp_path):
    cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cpu-mock"}}))
    assert cfg.deployment.profile == "cpu-mock"
    assert cfg.deployment.compute_target == "cpu_mock"
    assert cfg.indexing.detector_impl == "stub"
    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.storage.backend == "local"


def test_cloud_serving_profile(tmp_path):
    cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
    assert cfg.deployment.profile == "cloud-serving"
    assert cfg.deployment.compute_target == "cloud_run"
    assert cfg.indexing.detector_impl == "native"
    assert cfg.indexing.skip_ai_handoff is False  # Gemini-ON until the owned model clears the floor
    assert cfg.storage.backend == "gcs"


def test_truly_local_profile(tmp_path):
    cfg = load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
    assert cfg.deployment.profile == "truly-local"
    assert cfg.deployment.compute_target == "local_gpu"
    # detector_impl is intentionally left at the production default (env picks WSL vs native).
```

```python
    assert cfg.indexing.detector_impl == "auto"
    assert cfg.indexing.skip_ai_handoff is True  # zero-cloud
    assert cfg.storage.backend == "local"


# ---------------------------------------------------------------------------------------
# Precedence: defaults < preset < config.json (deep-merge layering).
# ---------------------------------------------------------------------------------------
def test_config_json_overrides_preset_subkey(tmp_path):
    """A file sub-key beats the preset's sub-key, but a sibling preset sub-key in the SAME
    section survives (this is the deep-merge layering ? a shallow merge would wipe it)."""
    cfg = load_config(_write(tmp_path, {
        "deployment": {"profile": "cpu-mock"},
        "indexing": {"detector_impl": "native"},  # override one preset knob...
    }))
    assert cfg.indexing.detector_impl == "native"     # file wins
    assert cfg.indexing.skip_ai_handoff is True       # ...preset sibling SURVIVES (deep merge)


def test_preset_does_not_wipe_unrelated_defaults(tmp_path):
    """Selecting a profile + setting an unrelated knob keeps the rest of the section at its
    built-in default (the preset and the file both layer onto the full defaults)."""
    cfg = load_config(_write(tmp_path, {
        "deployment": {"profile": "cpu-mock"},
        "indexing": {"ai_max_workers": 2},
    }))
    assert cfg.indexing.detector_impl == "stub"   # preset
    assert cfg.indexing.skip_ai_handoff is True   # preset
    assert cfg.indexing.ai_max_workers == 2       # file
    assert cfg.indexing.default_segmenter == "yolo_hybrid"  # untouched default
    assert cfg.indexing.max_window_duration == 6.0          # untouched default


def test_config_json_can_override_compute_target_with_warning(tmp_path, caplog):
    cfg_path = _write(tmp_path, {
        "deployment": {"profile": "cpu-mock", "compute_target": "cloud_run"},
    })
    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
        cfg = load_config(cfg_path)
    assert cfg.deployment.profile == "cpu-mock"
    assert cfg.deployment.compute_target == "cloud_run"  # explicit override honored (D15)
    assert any("overrides it to 'cloud_run'" in r.message for r in caplog.records)


# ---------------------------------------------------------------------------------------
# Env selection + precedence over config.json.
# ---------------------------------------------------------------------------------------
def test_env_selects_profile(tmp_path, monkeypatch):
    monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
    cfg = load_config(_write(tmp_path, {}))
    assert cfg.deployment.profile == "cpu-mock"
    assert cfg.indexing.detector_impl == "stub"


def test_env_profile_overrides_config_json_profile(tmp_path, monkeypatch):
    """env wins over config.json for the profile CHOICE, and the recorded profile matches the
    preset actually applied (not the file's value)."""
    monkeypatch.setenv("DEPLOYMENT_PROFILE", "cpu-mock")
    cfg = load_config(_write(tmp_path, {"deployment": {"profile": "cloud-serving"}}))
```

```
    assert cfg.deployment.profile == "cpu-mock"        # env value recorded
    assert cfg.deployment.compute_target == "cpu_mock"  # cpu-mock preset applied
    assert cfg.indexing.detector_impl == "stub"


def test_blank_env_profile_is_ignored(tmp_path, monkeypatch):
    monkeypatch.setenv("DEPLOYMENT_PROFILE", "   ")
    cfg = load_config(_write(tmp_path, {}))
    assert cfg.deployment.profile == ""
    assert cfg.indexing.detector_impl == "auto"


def test_unknown_env_profile_warns_and_applies_no_preset(tmp_path, monkeypatch, caplog):
    monkeypatch.setenv("DEPLOYMENT_PROFILE", "banana")
    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
        cfg = load_config(_write(tmp_path, {}))
    assert cfg.deployment.profile == ""          # not applied
    assert cfg.indexing.detector_impl == "auto"  # no preset
    assert any("unrecognized deployment profile 'banana'" in r.message for r in caplog.records)


def test_unknown_profile_in_config_json_raises(tmp_path, monkeypatch):
    """A bad VALUE in config.json hits the typed Literal ? a loud, hard error (same class as a
    bad `sport`), not a silent drop."""
    monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
    with pytest.raises(ValidationError):
        load_config(_write(tmp_path, {"deployment": {"profile": "banana"}}))


# ---------------------------------------------------------------------------------------
# D15 ? auto-detect SUGGESTS but never AUTO-APPLIES.
# ---------------------------------------------------------------------------------------
def test_cloud_env_does_not_auto_apply_profile(tmp_path, monkeypatch):
    """K_SERVICE (Cloud Run) makes suggest_profile() recommend cloud-serving, but with no
    explicit profile the loaded config stays fully default ? NO silent cloud selection."""
    monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
    monkeypatch.setenv("K_SERVICE", "khelsutra-serving")
    cfg = load_config(_write(tmp_path, {}))
    assert cfg.deployment.profile == ""           # NOT auto-applied
    assert cfg.deployment.compute_target == ""
    assert cfg.indexing.detector_impl == "auto"   # default, not "native"
    assert cfg.storage.backend == "local"         # default, not "gcs"


def test_active_profile_is_logged(tmp_path, caplog):
    with caplog.at_level(logging.INFO, logger="backend.config.loader"):
        load_config(_write(tmp_path, {"deployment": {"profile": "truly-local"}}))
    assert any("deployment profile ACTIVE: truly-local" in r.message for r in caplog.records)


def test_no_active_profile_info_when_unset(tmp_path, caplog, monkeypatch):
    monkeypatch.delenv("DEPLOYMENT_PROFILE", raising=False)
    with caplog.at_level(logging.INFO, logger="backend.config.loader"):
        load_config(_write(tmp_path, {}))
    assert not any("profile ACTIVE" in r.message for r in caplog.records)


# ---------------------------------------------------------------------------------------
# suggest_profile() ? pure env-only logic + the opt-in cuda probe.
# ---------------------------------------------------------------------------------------
```

```python
def test_suggest_profile_cloud_run():
    assert suggest_profile({"K_SERVICE": "svc"}) == "cloud-serving"


def test_suggest_profile_ci():
    assert suggest_profile({"CI": "true"}) == "cpu-mock"
    assert suggest_profile({"CI": "1"}) == "cpu-mock"
    assert suggest_profile({"CI": "yes"}) == "cpu-mock"


def test_suggest_profile_ci_falsy_or_absent():
    assert suggest_profile({}) == ""
    assert suggest_profile({"CI": "false"}) == ""
    assert suggest_profile({"CI": "0"}) == ""
    assert suggest_profile({"CI": ""}) == ""


def test_suggest_profile_k_service_beats_ci():
    assert suggest_profile({"K_SERVICE": "svc", "CI": "true"}) == "cloud-serving"


def test_suggest_profile_no_cuda_probe_by_default(monkeypatch):
    """probe_cuda defaults False ? no torch import, no truly-local suggestion even with a GPU."""
    import backend.config.presets as presets

    monkeypatch.setattr(presets, "_cuda_available", lambda: True)
    assert suggest_profile({}) == ""                     # probe off ? no suggestion
    assert suggest_profile({}, probe_cuda=True) == "truly-local"


def test_suggest_profile_cuda_probe_false(monkeypatch):
    import backend.config.presets as presets

    monkeypatch.setattr(presets, "_cuda_available", lambda: False)
    assert suggest_profile({}, probe_cuda=True) == ""


def test_cuda_available_never_raises(monkeypatch):
    """Even a broken/missing torch must degrade to False, never propagate."""
    import builtins

    import backend.config.presets as presets

    real_import = builtins.__import__

    def boom(name, *a, **k):
        if name == "torch":
            raise RuntimeError("no torch")
        return real_import(name, *a, **k)

    monkeypatch.setattr(builtins, "__import__", boom)
    assert presets._cuda_available() is False


# ---------------------------------------------------------------------------------
# Helpers + plumbing.
# ---------------------------------------------------------------------------------
def test_is_known_profile():
    assert is_known_profile("cpu-mock") is True
    assert is_known_profile("") is False
    assert is_known_profile("banana") is False


def test_preset_for_returns_copy():
```

```
    a = preset_for("cpu-mock")
    a["indexing"]["detector_impl"] = "mutated"
    b = preset_for("cpu-mock")
    assert b["indexing"]["detector_impl"] == "stub"  # module table not mutated
    assert preset_for("") == {}
    assert preset_for("banana") == {}


def test_save_default_config_includes_deployment(tmp_path):
    p = save_default_config(str(tmp_path / "out.json"))
    saved = json.loads(p.read_text())
    assert saved["deployment"] == {"profile": "", "compute_target": ""}
    cfg = load_config(str(p))
    assert cfg.deployment.profile == ""


def test_repo_config_json_has_no_profile():
    """The shipped config.json must not silently opt into a profile (default path stays today's
    behaviour for the real deployment)."""
    repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    cfg_path = os.path.join(repo_root, "config.json")
    if not os.path.exists(cfg_path):
        pytest.skip("no repo config.json")
    cfg = load_config(cfg_path)
    assert cfg.deployment.profile == ""

```

10. assistant (2026-06-21T07:24:08.803Z)

Now let me read the prior test contract and verify the existing knobs referenced by presets actually exist with the claimed defaults. Let me check the prior tests and verify against existing code.

11. user (2026-06-21T07:24:09.568Z)

```
"""Basic tests for the typed configuration package.

These ensure:
- Defaults are sensible and come from the models.
- File overrides work.
- Old config files with extra keys are tolerated.
- save_default_config produces loadable output.
"""
import json
import tempfile
import os


from backend.config import load_config, save_default_config, AppConfig


def test_load_default_config():
    # Force load without the project's config.json so we test the model built-in defaults
    cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
    assert isinstance(cfg, AppConfig)
    assert cfg.sport == "badminton"
    assert cfg.indexing.default_segmenter == "yolo_hybrid"
    assert cfg.validation.min_blur_threshold == 20.0  # updated default from review
    assert cfg.indexing.ai_max_workers == 5


def test_logging_config_default_quiets_http():
    """verbose_http defaults to False so run.log is quiet unless explicitly turned
```

```
up."""
    cfg = load_config("/nonexistent/path/12345/config.json")
    assert cfg.logging.verbose_http is False


def test_skip_ai_handoff_default_off():
    """Fully-local mode is opt-in: skip_ai_handoff defaults False so the AI handoff runs."""
    cfg = load_config("/nonexistent/path/12345/config.json")
    assert cfg.indexing.skip_ai_handoff is False


def test_skip_ai_handoff_file_override():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "skip.json")
        with open(p, "w") as fh:
            json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
        cfg = load_config(p)
        assert cfg.indexing.skip_ai_handoff is True


def test_logging_config_file_override():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "logging.json")
        with open(p, "w") as fh:
            json.dump({"logging": {"verbose_http": True}}, fh)
        cfg = load_config(p)
        assert cfg.logging.verbose_http is True


def test_fusion_config_defaults_reproduce_control():
    """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no mask.

    Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was REVERTED
    (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on the
    served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies). See
    docs/QUALITY_ITERATIONS.md and the served litmus in tests/test_served_gate_a_regression.py.
    """
    cfg = load_config("/nonexistent/path/12345/config.json")
    f = cfg.indexing.fusion
    assert f.substrate == "wasb"
    assert f.cue_flags == {}            # person cue OFF
    assert f.min_crossings == 0         # Gate A OFF (default) ? #146 served flip reverted
    assert f.min_players == 0           # Gate B OFF
    assert f.court_polygon is None      # no mask
    assert f.velocity_threshold == 0.02


def test_fusion_config_gate_a_opt_in_via_min_crossings():
    """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only operating
    point), while the served default stays OFF (#146 flip reverted)."""
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "optin.json")
        with open(p, "w") as fh:
            json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
        cfg = load_config(p)
        assert cfg.indexing.fusion.min_crossings == 2   # Gate A ON only when explicitly set
```

```


def test_served_default_segmenter_is_gemini():
    """The shipped config.json served default is the prior detector (gemini) ? the #146 flip
    to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
    repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    cfg = load_config(os.path.join(repo_root, "config.json"))
    assert cfg.indexing.default_segmenter == "gemini"
    # ...and the served fusion default keeps Gate A OFF (no served gating).
    assert cfg.indexing.fusion.min_crossings == 0


def test_fusion_config_file_override():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "fusion.json")
        with open(p, "w") as fh:
            json.dump({"indexing": {"fusion": {
                "substrate": "tracknet", "min_crossings": 2,
                "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1], [0.5, 0.9]],
            }}}, fh)
        cfg = load_config(p)
        f = cfg.indexing.fusion
        assert f.substrate == "tracknet"
        assert f.min_crossings == 2
        assert f.cue_flags == {"person": True}
        assert len(f.court_polygon) == 3
        # An old config without a fusion block still loads (defaults applied).
        assert cfg.indexing.wasb.velocity_threshold == 0.02


def test_file_override():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "override.json")
        data = {
            "indexing": {
                "default_segmenter": "gemini",
                "ai_max_workers": 3,
            },
            "validation": {
                "min_blur_threshold": 42.0,
            },
        }
        with open(p, "w") as f:
            json.dump(data, f)

        cfg = load_config(p)
        assert cfg.indexing.default_segmenter == "gemini"
        assert cfg.indexing.ai_max_workers == 3
        assert cfg.validation.min_blur_threshold == 42.0
        # Unspecified keys fall back to defaults
        assert cfg.sport == "badminton"


def test_extra_keys_tolerated():
    """Old or future keys in config.json must not break loading."""
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "with_extra.json")
        data = {
            "sport": "badminton",
            "indexing": {"default_segmenter": "yolo_hybrid"},
            "some_future_key": "ignored_for_now",
            "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
        }
```

```
        with open(p, "w") as f:
            json.dump(data, f)

        cfg = load_config(p)
        assert cfg.indexing.default_segmenter == "yolo_hybrid"
        assert cfg.validation.relevance.court_pixels_min_ratio == 0.1


def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
    """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
    section keys are silently dropped, so without this warning a typo just makes
    the knob a no-op (the audit's 'silently dropped config' class)."""
    p = tmp_path / "typo.json"
    p.write_text(json.dumps({
        "some_future_key": 1,                                   # top-level extra
        "indexing": {"velocty_threshold": 0.5},                 # typo'd section key
        "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
    }))
    import logging
    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
        cfg = load_config(str(p))
    assert isinstance(cfg, AppConfig)  # still non-fatal
    joined = " ".join(r.message for r in caplog.records)
    assert "some_future_key" in joined
    assert "indexing.velocty_threshold" in joined
    assert "validation.relevance.unknown_gate" in joined


def test_recognized_config_does_not_warn(tmp_path, caplog):
    p = tmp_path / "clean.json"
    p.write_text(json.dumps({
        "sport": "badminton",
        "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold": 0.02}},
    }))
    import logging
    with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
        load_config(str(p))
    assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]


def test_non_object_config_degrades_to_defaults(tmp_path, caplog):
    """A config.json that is valid JSON but NOT an object (`[]`, `[1]`, `"x"`, `42`, `true`,
    `null`) must degrade to a fully-defaulted AppConfig + a warning ? never crash startup.
    Regression: previously a truthy non-mapping hit `.items()` (AttributeError) and a falsy one
    broke the `{**defaults, **file_data}` unpack (TypeError), turning an operator typo into a hard
    server/CLI startup crash."""
    import logging
    default_dump = AppConfig().model_dump(mode="json")
    for raw in ("[]", "[1, 2]", '"x"', "42", "true", "null"):
        p = tmp_path / "bad.json"
        p.write_text(raw)
        caplog.clear()
        with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
            cfg = load_config(str(p))               # must not raise
        assert isinstance(cfg, AppConfig)
        assert cfg.model_dump(mode="json") == default_dump, f"{raw} should yield pure defaults"
        assert any("not a JSON object" in r.message for r in caplog.records), \
            f"{raw} should log the non-object warning"

```

```python

def test_save_default_config_roundtrip():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "saved.json")
        saved = save_default_config(p)
        assert str(saved) == p
        assert os.path.exists(p)

        cfg = load_config(p)
        assert cfg.indexing.default_segmenter == "yolo_hybrid"
        assert cfg.validation.min_blur_threshold == 20.0


def test_model_is_the_source_of_truth():
    """Changing a default in the model should be reflected without touching json."""
    # This is a documentation test more than anything.
    cfg = AppConfig()
    assert cfg.indexing.tracknet.velocity_threshold == 0.02


def test_output_dir_field_default_and_override():
    """output_dir now lives on OutputConfig (was a write-only runtime hack), so
    /api/compile and the CLI resolve the same value. Default + file override + the
    runtime mutation that backend/main.py::main performs for --output-dir."""
    cfg = AppConfig()
    assert cfg.output.output_dir == "output"  # built-in default

    # The CLI override path: main() assigns args.output_dir onto the typed model.
    cfg.output.output_dir = "/tmp/custom_out"
    assert cfg.output.output_dir == "/tmp/custom_out"

    # File override is honored on load.
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "out_override.json")
        with open(p, "w") as f:
            json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
        loaded = load_config(p)
        assert loaded.output.output_dir == "reels"
        assert loaded.output.db_name == "x.db"

def test_config_shallow_section_replacement():
    """Positive test for Item 2 (config loader): shallow section replacement.

    File-provided 'indexing' section replaces the defaults 'indexing' section entirely
    (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
    """
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "shallow.json")
        with open(p, "w") as fh:
            json.dump({"indexing": {"default_segmenter": "motion"}}, fh)
        cfg = load_config(p)
        assert cfg.indexing.default_segmenter == "motion"
        # Pydantic defaults fill the rest of the (replaced) section
        assert cfg.indexing.ai_max_workers == 5


def test_config_unknown_key_paths_warning_and_tolerate():
    """Positive test for Item 2: _unknown_key_paths warning for unknown top-level keys,
    but tolerated (extra='allow'), and unknown section keys dropped silently.
    """
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "unknown.json")
        with open(p, "w") as fh:
            json.dump({
                "indexing": {"default_segmenter": "motion"},
```

```
                "completely_unknown_top": "bar",
                "validation": {"unknown_in_section": 99}  # dropped
            }, fh)
        cfg = load_config(p)
        assert cfg.indexing.default_segmenter == "motion"
        # tolerated, no crash


def test_get_returns_dict_for_nested_sections_legacy_compat():
    """Regression: legacy consumers (validators/segmenters) do
    ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
    .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
    chained access raises 'X object has no attribute get' on every real run.

    This is the original strong legacy test (restored per #160 review). It covers the
    full set of footgun guards: dict sections, nested dicts, scalar leaves with defaults,
    missing key fallbacks, and leaf keys discoverable inside sections.
    """
    cfg = load_config("/nonexistent/123/config.json")
    validation = cfg.get("validation", {})
    assert isinstance(validation, dict)
    assert validation.get("min_resolution_width") == 1280
    # nested-of-nested is also dict-accessible
    assert isinstance(validation.get("relevance"), dict)
    assert validation["relevance"].get("court_pixels_min_ratio") == 0.05

    # scalar leaves are returned unchanged; a present scalar ignores the default arg;
    # a missing key falls back to the default
    assert cfg.get("ai_provider") == "gemini"
    assert cfg.get("ai_provider", "ignored-default") == "gemini"
    assert cfg.get("does_not_exist", "fallback") == "fallback"

    # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
    # list), so a future top-level section's keys would be discoverable too
    assert cfg.get("min_resolution_width") == 1280  # lives inside `validation`
    assert cfg.get("begin_padding") == 0.5          # lives inside `stitching`

    # legacy chained access still works
    assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"


def test_config_roundtrip_and_typed_reaches_pipeline(monkeypatch):
    """Positive for Item 2: save_default_config + load_config round-trips;
    typed AppConfig is what reaches pipeline code (e.g. main uses global_config as AppConfig).
    """
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "roundtrip.json")
        save_default_config(p)
        cfg = load_config(p)
        assert isinstance(cfg, AppConfig)
        assert cfg.sport == "badminton"

    # positive that typed is used in pipeline code (global_config is set at startup in main();
    # monkeypatch here so the test does not depend on server initialization)
    import backend.main as main_module
    from backend.main import get_config
    monkeypatch.setattr(main_module, "global_config", cfg)
    assert isinstance(get_config(), AppConfig)

```

12. assistant (2026-06-21T07:24:16.100Z)

I have a key finding emerging. The prior test `test_config_shallow_section_replacement` (test_config.py:246-259) documents that the no-profile path does shallow section replacement. The new code preserves that for the no-profile path but switches to deep-merge for the profile path. Let me verify there's no contradiction, and confirm the critical M1 spec concerns. Let me verify the existing knobs referenced by presets exist (`detector_impl`, `skip_ai_handoff`, `storage.backend` values, "native", "stub", "gcs").

The models confirm:
- `indexing.detector_impl` default "auto", accepts "native"/"stub"/"auto" (str, models.py:133)
- `indexing.skip_ai_handoff` default False (models.py:200)
- `storage.backend` Literal["local","s3","gcs","gdrive"] (models.py:302) ? "gcs" and "local" are valid

Now let me verify the critical M1-scope claims about what the presets do NOT touch ? confirm `input_backend` doesn't exist yet (M2) and that `single_folder_per_video`/output base are not touched. Let me grep.

13. user (2026-06-21T07:24:17.656Z)

```
Found 1 file
backend\config\presets.py
```

14. user (2026-06-21T07:24:18.472Z)

```
backend\config\models.py:339:    compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend\config\presets.py:45:        "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:58:        "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\loader.py:156:            ct = merged["deployment"].get("compute_target")
backend\config\loader.py:160:                    "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:164:            logger.info("[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:165:                        profile,
merged["deployment"].get("compute_target"))
```

15. assistant (2026-06-21T07:24:23.778Z)

`input_backend`/`input_root` appear only in `presets.py` (the comment, not as a knob) ? confirmed not set. `compute_target` is only read in the loader for logging/warning and nowhere in the core ? confirmed inert in M1. The grep on `input_backend` matched only `presets.py` which is the comment at line 39. Let me verify the presets dict itself does NOT contain `input_backend` or `single_folder_per_video` or `output_root` as actual keys (I read the file ? it doesn't).

Now let me confirm the deep-merge for the profile path doesn't accidentally change behavior of the shallow no-profile path, and verify the precedence chain via the actual test suite. Let me run the config tests.

16. user (2026-06-21T07:24:33.523Z)

```
....................................................                     [100%]
52 passed in 6.92s
```

17. assistant (2026-06-21T07:24:45.663Z)

All 52 tests pass. Now let me carefully scrutinize the precedence/deep-merge logic for the env-vs-config.json case and a subtle concern: the spec says precedence is `built-in defaults < profile preset < config.json < env`. In the new loader, the deep-merge is `_deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)`. This means file_data (config.json) wins over preset, which wins over defaults. Correct.

But there's a subtlety about `compute_target` when env selects a profile but config.json sets a different profile's compute_target. Let me trace one specific scenario carefully ? env selects `cpu-mock`, config.json has `{"deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"}}`.

Let me trace: `profile = "cpu-mock"` (env wins). `merged = _deep_merge(_deep_merge(defaults, preset_for("cpu-mock")), file_data)`. The preset sets `deployment.compute_target = "cpu_mock"`, but file_data has `deployment.compute_target = "cloud_run"` which deep-merges and wins. Then `merged["deployment"]["profile"] = "cpu-mock"` (re-stamped). So final: `profile=cpu-mock, compute_target=cloud_run`. The warning logic compares `ct=cloud_run` vs `canonical=cpu_mock` ? warns. That's coherent behavior.

Now let me check a potential real concern: does the no-profile path still correctly suggest without applying? And is there any scenario where `suggest_profile()` in the no-profile branch (loader.py:169) could read `K_SERVICE` and cause a side effect? It only logs at debug level ? no application. Confirmed inert.

Let me verify one more thing ? D11 says cloud-serving keeps Gemini ON. The preset sets `skip_ai_handoff: False`. But `skip_ai_handoff: False` is also the default. Let me confirm this is coherent and check whether `detector_impl: native` correctly reflects the spec. Let me check the `__init__.py` re-exports don't leak anything and verify the full broader config-consuming suite isn't broken.

18. user (2026-06-21T07:24:47.159Z)

f518e97 docs(serving): M0 APPROVED + lock D13-D15 (switchable profiles, Drive Northstar, consent-not-surprise) (#278)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness (M11) (#277)
f592281 docs: point I18N_PLAN + UI_PROPOSAL (+ DOC_STATUS) at the khelsutra UX/l10n project (#275)
---branch---
feat/serving-m1-deployment-profile

19. user (2026-06-21T07:24:47.895Z)

Found 3 files
backend\config\presets.py
backend\config\loader.py
backend\config\models.py

20. assistant (2026-06-21T07:24:58.274Z)

`deployment.profile` and `compute_target` are read ONLY in the config package ? nowhere in the core (no segmenter, no detector, no stitcher, no main/API branches on them). This confirms claim (d): compute_target is inert/recorded-only in M1.

Let me do a final check on one subtle correctness issue: the `_deep_merge` precedence with env-selected profile when config.json carries an `indexing` block. The re-stamping at loader.py:153-155 uses `merged.setdefault("deployment", {})`. Since the profile path always has `deployment` in the preset, this is fine. Let me also double-check the no-profile path against the prior `test_config_shallow_section_replacement` contract ? that test still passes (it was in the 52).

Now let me verify the one thing that could be an over-reach concern: does the `cpu-mock` preset's `storage.backend: local` and `truly-local`'s `storage.backend: local` match

D-decisions, and is `cloud-serving`'s `gcs` correct? Per §4.2 table: truly-local ? `local`, cloud-serving ? `gcs`, cpu-mock ? `local`. All match exactly.

Let me also verify nothing in the preset touches `single_folder_per_video`, `output_root`, or `workspace_root` (M3 gated). From reading presets.py, the `storage` key only contains `{"backend": ...}`. Confirmed ? no M3 knobs touched.

One last verification: confirm `detector_impl` values "native"/"stub" are accepted by the model (it's a plain `str` field at models.py:133, so any string validates) and that these are the real impl names. Let me confirm the detector impls exist.

21. user (2026-06-21T07:24:59.467Z)

```
backend\pipeline\detectors\native_wasb_runner.py:22:the windowing brain are unchanged. Selected
via the existing ``detector_impl`` seam
backend\pipeline\detectors\native_wasb_runner.py:23:(``trajectory_hybrid._resolve_runner`` ?
``_build_native_runner``); ``detector_impl="native"``.
backend\pipeline\detectors\stub_runner.py:20:``indexing.detector_impl="stub"`` or the
``RALLY_DETECTOR_IMPL=stub`` env var; the real
backend\pipeline\detectors\stub_runner.py:51:        s = (idx_cfg or {}).get("stub", {}) or {}
backend\pipeline\detectors\stub_runner.py:102:        key = f"{stem}__{video_id[:12]}" if
video_id else (stem or "stub")
```

22. assistant (2026-06-21T07:25:02.597Z)

Let me verify how `detector_impl` is consumed by `_resolve_runner` to confirm the env var override (`RALLY_DETECTOR_IMPL`) genuinely sits above config.json in precedence ? a claim in §4.3 that the loader relies on being enforced downstream.

23. user (2026-06-21T07:25:04.185Z)

```
61:            f"detector_impl='native' is not supported for the "
63:            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
65:    def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
68:        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
76:        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
79:            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
81:            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
83:            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
190:        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191:        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
194:            runner, runner_params = self._resolve_runner(idx_cfg)
196:            # e.g. detector_impl="native" for a family without a native runner (tracknet, or
```

24. assistant (2026-06-21T07:25:16.759Z)

This confirms the precedence claim: `RALLY_DETECTOR_IMPL` env var takes precedence over `indexing.detector_impl` (trajectory_hybrid.py:76), which the preset sets. So `built-in defaults < profile preset < config.json < env` holds for `detector_impl` because the preset value lands in `indexing.detector_impl` and the env var overrides it at consumption time. Consistent with §4.3.

I have thoroughly reviewed the implementation against the spec. Let me consolidate my assessment of each lens criterion:

(a) Auto-detect SUGGESTS, never auto-applies ? `_resolve_explicit_profile` (loader.py:47-59) consults ONLY `DEPLOYMENT_PROFILE` env and `config.json deployment.profile`, NOT `suggest_profile`. In the no-profile branch (loader.py:166-175), `suggest_profile()` is called but its result is only logged at debug, never applied. `test_cloud_env_does_not_auto_apply_profile` proves a bare `K_SERVICE` stays fully default. CONFIRMED clean.

(b) Presets only touch existing knobs, no M2/M3 pre-emption ? presets touch only `deployment.{profile,compute_target}`, `indexing.{detector_impl,skip_ai_handoff}`, `storage.backend`. No `input_backend` (M2), no `single_folder_per_video`/`output_root`/`workspace_root` (M3). CONFIRMED clean.

(c) Preset values coherent with decisions ? cloud-serving: `detector_impl=native`, `skip_ai_handoff=False` (Gemini ON per D11), `storage=gcs`; truly-local: `skip_ai_handoff=True` (zero-cloud), `storage=local`; cpu-mock: `detector_impl=stub`, `skip_ai_handoff=True`, `storage=local`. All match §4.2 + D11. CONFIRMED clean.

(d) compute_target inert in M1 ? read ONLY in the config package (loader logging/warning); no core code branches on it. CONFIRMED clean.

No bugs, no over-reach, no decision contradictions. The default path is byte-identical (`test_default_config_dump_only_adds_deployment_section` proves the dump equals a fresh `AppConfig()` plus the empty deployment section). Tests are green (52 passed). This is a clean bill.

I will note one genuinely minor observation for completeness, but it does not rise to a finding worth flagging as a defect: the `truly-local` preset deliberately omits `detector_impl` (leaves it at `auto`), which is documented and correct per §4.2 ("WSL?auto"). This is intentional and tested.

25. user (2026-06-21T07:25:17.016Z)

Structured output provided successfully

26. assistant (2026-06-21T07:25:32.721Z)

Clean bill ? no findings. M1 adheres to the spec and D15.

What I verified against README §4.3 and §7 M1, grounded in code:

(a) Auto-detect SUGGESTS, never auto-applies (D15) ? `_resolve_explicit_profile` (`backend/config/loader.py:47-59`) consults ONLY `DEPLOYMENT_PROFILE` env + `config.json deployment.profile`; it never calls `suggest_profile`. In the no-profile branch (`loader.py:166-175`), `suggest_profile()` runs but its result is only logged at `debug`, never applied. A bare `K_SERVICE`/`CI`/cuda box therefore selects nothing ? proven by `test_cloud_env_does_not_auto_apply_profile` (config stays fully default: `profile=""`, `detector_impl="auto"`, `backend="local"`).

(b) No M2/M3 pre-emption ? the presets (`backend/config/presets.py:43-63`) touch only knobs that exist today: `deployment.{profile,compute_target}`, `indexing.{detector_impl,skip_ai_handoff}`, `storage.backend`. No `input_backend` (M2 ? confirmed it exists nowhere in `backend/` except the explanatory comment at `presets.py:39`), and no `single_folder_per_video`/`output_root`/`workspace_root` (M3, Launch-Report-gated). The `storage` preset key carries only `{"backend": ...}`.

(c) Preset values coherent with decisions ? matches §4.2 + D11 exactly: cloud-serving ? `detector_impl=native`, `skip_ai_handoff=False` (Gemini ON), `storage.backend=gcs`; truly-local ? `skip_ai_handoff=True` (zero-cloud), `backend=local`, `detector_impl` left at `auto` (env picks WSL/native, per §4.2); cpu-mock ? `detector_impl=stub`, `skip_ai_handoff=True`, `backend=local`. `storage.backend` literals (`local`/`gcs`) are valid per `models.py:302`.

(d) compute_target inert in M1 ? read ONLY inside the config package (loader.py:156-165, for the override warning + the ACTIVE log line); a repo-wide grep shows nothing in the core (segmenters/detectors/stitcher/main/API) branches on `deployment.*`.

Purity/additivity ? `test_default_config_dump_only_adds_deployment_section` proves the loaded default dump equals a fresh `AppConfig()` dump (default path byte-identical). The new deep-merge is scoped to the profile path only; the no-profile path keeps the original shallow `{**defaults, **file_data}` merge (its `test_config_shallow_section_replacement` contract still passes). Precedence `defaults < preset < config.json < env` holds ? the env-over-config.json layer for `detector_impl` is enforced downstream at `trajectory_hybrid.py:76` (`RALLY_DETECTOR_IMPL` over `indexing.detector_impl`).

Test suite green: `tests/test_config_deployment.py` + `tests/test_config.py` ? 52 passed.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/presets.py`,
`C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/loader.py`,
`C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py`,
`C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config_deployment.py`.